Research Note: Rethinking Low-Level Design (LLD) Practice

1. The Learner Problem in Low-Level Design

Mastering Low-Level Design (LLD) / Object-Oriented Design (OOD) is one of the steepest hurdles for software engineers targeting senior and staff roles. Unlike Data Structures & Algorithms (DSA), where correctness is binary (an algorithm either passes edge cases within time limits or fails), LLD lives in an inherently nuanced space:

- **Multiple valid designs exist**: A Parking Lot can be designed with a Strategy pattern, a State pattern, or an event-driven observer model.
- **Subjectivity vs. Fundamentals**: While trade-offs are subjective, core object-oriented principles (Single Responsibility Principle, encapsulation, explicit entity boundaries, avoiding God classes) are not.
- **Lack of immediate feedback**: When an engineer practices an LLD problem alone on a whiteboard or local IDE, there is no compiler or unit test harness to tell them: "*You concentrated billing, gate mechanics, and spot allocation in a 300-line God class*" or "*You forgot to model a ParkingTicket entity and passed around raw timestamps.*"

Without rapid, explainable feedback, learners either over-engineer complex design patterns where they do not belong or write procedural scripts masquerading as object-oriented code.

---

2. Existing Approaches & Their Gaps

Existing Approach	Strengths	Critical Gaps in LLD Practice
LeetCode / HackerRank	Instant automated feedback, zero scheduling friction, scalable scoring.	**Zero architectural critique.** Only tests runtime I/O against test cases. Encourages monolithic script-style solutions in a single `Solution` class that would fail any real LLD interview.
Mock Interview Platforms *(Interviewing.io, Pramp)*	Real human conversational dynamics, authentic pressure simulation.	**High cost & friction.** Expensive ($100–$250/session), requires scheduling days in advance, and feedback quality varies wildly depending on the assigned interviewer's seniority.
Peer Review Communities *(Discord, Reddit, GitHub)*	Free, community-driven, diverse viewpoints.	**Low throughput & inconsistent standards.** Majority of posted code reviews receive zero replies; feedback received is often contradictory, delayed by days, and lacks standardized rubrics.
Generic LLMs *(ChatGPT, Claude raw chat)*	Readily available, conversational, broad knowledge base.	**Sycophantic & ungrounded.** Raw LLM chats tend to praise suboptimal code ("Looks great! Here is a minor tip..."), hallucinate requirements, lack deterministic checks, and fail to track structured progress over multiple attempts.

---

3. The Core Gaps We Target

Our platform targets three specific structural gaps:

1. **The Grounded Hybrid Feedback Gap**: Relying purely on LLMs causes hallucinations and vague feedback; relying purely on unit tests tests runtime input/output rather than object-oriented structure. We bridge this with a **two-tier evaluation engine**:
   - *Deterministic Static AST Analysis*: Directly inspects the Python Abstract Syntax Tree to verify domain entities, verify core method APIs, flag Single Responsibility Principle (SRP) violations (God classes), and check composition relationships with concrete line citations.
   - *Qualitative Architectural Review*: Pluggable LLM interface that reviews design pattern applicability (e.g. Strategy vs Factory), extensibility, concurrency considerations, and trade-offs.
2. **The Iterative Refinement Gap**: Real design learning occurs when refactoring from a naive draft to an extensible architecture. Most platforms treat solving as a single transaction. We model **Attempt History and Delta Progression**:
   - Comparing Attempt $N-1$ to Attempt N.
   - Explicitly showing resolved issues (e.g., "*Resolved God class in `ParkingLot`*"), persisting issues, and score improvements.
3. **The Clear Scope Boundary Gap**: Candidates frequently fail LLD rounds because they lose 30 minutes designing SQL schemas, IoT sensor communication, or payment gateway network retries instead of class relationships. We provide explicit **Scope Boundaries** per problem (e.g., "*Out of scope: Payment gateway processing; focus on spot allocation and fee calculation strategy*").

---

4. Product Direction & MVP Scope

Our MVP intentionally focuses on the core learner loop:

```
Pick Problem → Read Boundaries → Submit Design → Receive Grounded Feedback → Review Progression → R
```

Key Decisions for the MVP:

- **Language & Submission Format**: Python AST-parseable class skeletons and implementations. Candidates write actual classes, methods, and type hints. Python's standard `ast` module enables zero-dependency, instantaneous, and deterministic analysis.
- **Architectural Simplicity**: A clean, modular monolith with zero external infrastructure dependencies. No Docker, no Kubernetes, no external database requirements.
- **Explainability First**: Every feedback item must cite specific entities, line numbers, and actionable suggestions.